

NVIDIA CUDA vs. CPU Computing: A Comparative Analysis of Parallel Processing for High-Performance Data Workloads

Tan Zhi Ming

University Technology Malaysia

tanzhiming@graduate.utm.my

Abstract—The increasing number of data-intensive applications has created a need for high-performance computing platforms capable of effectively running large-scale workloads. General-purpose computing still requires traditional CPU-based systems, which are typically limited in throughput for massively parallel tasks. NVIDIA CUDA has become a significant platform to enable general-purpose computation on graphics processing units (GPUs), enabling developers to harness large-scale parallelism for scientific computing, machine learning, and other data-intensive applications. This paper discusses the architecture and programming model of CUDA and compares CUDA-enabled GPU computing with traditional CPU computing. The discussion notes disparities in processing philosophy, architectural design, and workload appropriateness. It also examines two application areas, deep learning and molecular modeling, to illustrate the real-world impact of CUDA on contemporary computing. The paper concludes that CUDA-based GPU computing offers significant performance benefits for highly parallel workloads, especially those with high arithmetic intensity and task independence; however, CPUs remain more suitable for sequential, irregular, control-intensive, or reuse-heavy workloads. By clarifying these trade-offs, this comparison helps readers decide when CUDA should be deployed as a powerful accelerator and when CPU computing remains the more appropriate solution.

Index Terms—NVIDIA CUDA, GPU computing, CPU computing, high-performance computing

I. INTRODUCTION

The importance of high-performance data processing has grown due to the need for modern computing systems to process larger data volumes, complex simulations, and train machine learning models within realistic time constraints. The architecture of traditional CPU-based systems remains the basis of general-purpose computing, although they are generally inefficient for workloads with high data parallelism. Conversely, GPU computing has evolved into a significant means of accelerating highly parallel operations by increasing arithmetic throughput and memory bandwidth (Owens et al., 2008).

Before CUDA, GPUs were mainly designed and used for graphics processing, while most general-purpose computation still depended on CPUs. However, as scientific computing, simulation, and data-intensive applications became larger, CPUs alone could not always provide enough throughput for workloads that involved many repeated parallel operations. At the same time, researchers had begun to recognize that GPUs had strong computational potential beyond graphics because

of their high arithmetic throughput and memory bandwidth. NVIDIA introduced CUDA to make this GPU capability more accessible to developers by allowing general-purpose programs to be written for NVIDIA GPUs using a scalable parallel programming model based on threads, thread blocks, and grids (Nickolls et al., 2008). As a result, CUDA helped transform the GPU from a graphics-specific device into a practical parallel computing platform for scientific computing, machine learning, and simulation.

This issue is relevant since the current high-performance data loads are more and more relying on architectures capable of performing many operations concurrently. However, the value of CUDA can only be understood properly through comparison with CPU computing. In this context, the distinction between GPU and CPU architectures is central to evaluating their suitability for different workloads.

Thus, the purpose of this paper is to compare NVIDIA CUDA and conventional CPU computing from the perspective of architectural design, programming model, workload suitability, and practical performance in high-performance data processing. It begins with a literature review on the topics of GPU architectures and CUDA, and then covers two subtopics: the first is the architectural and performance comparison of CUDA-enabled GPU computing to CPU computing; the second is the use of CUDA in deep learning and scientific computing. In this discussion, the paper assesses the strengths and weaknesses of CUDA in high-performance data processing.

II. LITERATURE REVIEW

The literature on GPU computing consistently shows that the push for general-purpose GPU computation came from a desire to boost throughput in data-parallel tasks. Owens et al. (2008) introduced GPU computing as a shift from specialized graphics hardware to programmable, high-performance processors. These processors offer higher arithmetic throughput and memory bandwidth than CPUs. They also reported successful uses of GPUs in scientific and computational fields. Therefore, by the end of the 2000s, GPU computing was no longer limited to graphics but had become an important approach for accelerating data-intensive and high-performance computing tasks (Owens et al., 2008).

Owens et al. (2007) surveyed how researchers began using graphics processors for non-graphics problems. This was due to their high computational density and low cost. The survey also reported significant constraints, such as programming difficulties and limitations with irregular calculations. Architectural constraints from graphics-focused design also existed. This literature is valuable because it shows GPU computing as highly efficient for only some parallel problems, not universally superior. This finding is supported by Fatahalian et al. (2004), who showed that even highly parallel matrix-matrix multiplication did not always perform better on GPUs than on optimized CPUs when memory access and data reuse were not handled efficiently.

The creation of CUDA was a significant move towards making GPU computation more approachable. Nickolls et al. (2008) described that CUDA presented a thread-block, thread-grid model, along with shared memory and synchronization primitives. These abstractions allowed parallel programs to be expressed directly, without graphics APIs or workarounds based on shaders. CUDA thus aided in the creation of a more realistic and scalable model of general-purpose GPU computing.

The other significant literature is that of architectural comparison. Garland and Kirk (2010) explained that, unlike CPUs, GPUs are throughput-oriented architectures. To them, the design of the GPU is premised on the assumption that there is plenty of parallelism and is particularly well suited to highly parallel scientific and simulation workloads. This view gives the theoretical foundation to the comparison of the performance of the GPUs and the CPU in high-performance data processing.

Meanwhile, the literature also cautions against simplified arguments concerning the superiority of GPUs. In their study of matrix-matrix multiplication, Fatahalian et al. (2004) demonstrated that even optimized GPU implementations might still be less efficient than more cache-aware CPU implementations of certain reuse-intensive computations. Their results are meaningful since they prove that the acceleration of GPUs is not predetermined by the highest arithmetic performance only; the patterns of memory access and the efficiency of the cache also have a decisive impact.

Lastly, there are application studies that demonstrate the impact of CUDA on contemporary computing. Krizhevsky et al. (2012) showed that with the help of GPU acceleration, it was possible to train large convolutional neural networks on the ImageNet dataset, which resulted in a significant breakthrough in the performance of deep learning. Likewise, Stone et al. (2007) demonstrated that CUDA-based acceleration of GPUs could be of great benefit to molecular modeling and electrostatic computations, typically one to two orders of magnitude faster than optimized CPU implementations.

In general, the literature reviewed can be summarized into three general conclusions. To start with, CUDA makes the practicality of GPU programming significantly better. Second, GPUs are faster than CPUs in most highly parallel workloads due to their throughput-oriented architecture. Third, the advan-

tages of CUDA are workload-specific and cannot be assumed to be universal..

III. ARCHITECTURAL AND PERFORMANCE ANALYSIS

The fundamental distinction between CUDA-based GPU computing and conventional CPU computing is in architectural philosophy. CPUs are mainly developed to reduce latency in the sense that they strive to accomplish single or sequential tasks in the shortest time possible. They usually do this by having fewer cores that are more powerful, large caches, and complex control systems. In comparison, GPUs are optimized towards throughput by performing very large numbers of simpler operations in parallel. Garland and Kirk (2010) clearly defined this differentiation as latency-oriented and throughput-oriented computing.

The significance of CUDA is that it offers an effective way of tapping into this throughput-based architecture. CUDA enables a problem to be broken down into numerous smaller tasks that can be executed simultaneously on the GPU through its hierarchy of threads, blocks, and grids. This model of execution is particularly appropriate when workloads have a repetitive mathematical operation that needs to be performed on a large number of data items. Nickolls et al. (2008) explained that this design makes CUDA particularly effective for scalable parallel programming.

In terms of performance, GPU computing is often more efficient than CPU computing in cases where the task is highly parallel and arithmetic-intensive. According to Owens et al. (2008), GPU computing has been shown to provide much higher arithmetic throughput and memory bandwidth than traditional CPUs on a wide range of workloads, and is especially appealing to large-scale numerical computation. Moreover, Stone et al. (2007) demonstrated that molecular modeling kernels, which are implemented on GPUs, can be 10-100 times faster than highly optimized CPU implementations, depending on the job.

Figure 1 shows the hierarchical structure of a contemporary NVIDIA GPU. Fast local resources (registers and shared memory) are found in each streaming multiprocessor (SM), with all SMs accessing larger data via L2 cache and global memory. This architecture is intended to enable high-throughput execution of a large number of concurrent threads, one of the most important architectural distinctions between GPU computing and conventional CPU computing.

However, the comparison is not one-sided. In their study, Fatahalian et al. (2004) showed that even when obviously parallel, matrix-matrix multiplication was not always more efficient on GPUs than on well-optimized CPUs. This was not due to the inability to perform arithmetic, but to the inadequacy of high-bandwidth access to data in the cache for reuse-intensive computation. This observation provides important evidence that GPUs are not necessarily better than CPUs in all numerical tasks, because performance also depends on memory access patterns, cache efficiency, and data reuse (Fatahalian et al., 2004).

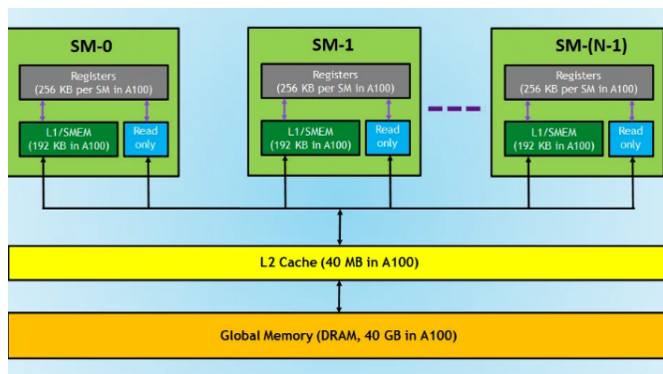


Fig. 1. Simplified memory hierarchy of a modern NVIDIA GPU architecture. Reprinted from “CUDA Refresher: CUDA Programming Model,” by P. Gupta, 2020, NVIDIA Developer Blog.

An equal comparison can thus be drawn in various dimensions. In the processing model, CPU computing is still more powerful in sequential or irregular work, and CUDA-enabled GPU computing is more powerful in large-scale data-parallel workloads. The CPU development is usually easier in a programming model, whereas in CUDA, thread decomposition, synchronization, and memory hierarchy are to be explicitly considered. CPUs are optimized by cache-oriented behavior in memory, whereas GPUs are more bandwidth-constrained and require regular access behavior. CPUs are more general-purpose computing, but CUDA-based GPUs are best in highly parallel, repetitive, and computation-intensive applications.

Overall, this section demonstrates the usability of CUDA in accelerating workloads with high parallelism and arithmetic intensity, while also showing that CPUs remain more suitable for less parallel, irregular, and control-intensive tasks.

CPU and CUDA-based GPU computing have different performance objectives as indicated in Table 1. CPUs are still more appropriate to sequential and irregular workloads, and CUDA-enabled GPUs are more appropriate to highly parallel and arithmetic-intensive workloads. This is the reason why CUDA has gained relevance in deep learning and scientific computing, and CPUs still dominate most of the general-purpose tasks.

IV. DEPLOYMENT OF CUBA

The usefulness of CUDA is most evident in real-world applications. One of the strongest examples is deep learning. Krizhevsky et al. (2012) trained a large convolutional neural network on ImageNet with GPU acceleration and obtained a top-5 error rate of 15.3%, in comparison with the second-best result of 26.2% in the competition. Their contribution is generally considered a breakthrough in the field of modern artificial intelligence since it demonstrated that deep neural networks could be efficiently trained at scale with the help of GPU acceleration.

This success was directly connected with the workload structure. Deep learning training consists of repeated matrix multiplications, convolutions, and vector operations with very

large datasets. These are inherently parallelizable operations, i.e., they can be spread among a large number of GPU threads. In this regard, CUDA enabled the conversion of the hardware of the graphics platform to a practical training platform for modern neural networks. Consequently, CUDA was not merely a performance incremental improvement, but it enabled a new generation of large-scale AI models to be created.

A second key area of application is scientific computing. Stone et al. (2007) presented solid arguments that CUDA-based GPU computing can significantly speed up molecular modeling programs, such as electrostatic potential calculation, Coulomb summation, and molecular dynamics force calculations. Their article documented that GPU-based computation is generally 10-100 times quicker than highly optimized CPU-based code and also mentioned particular CUDA kernels that had significant speedups. The significance of these findings is that they demonstrate that CUDA is not restricted to commercial machine learning applications; it is also quite useful in computational science, where large-scale simulations may be very slow on CPUs alone.

These areas of application are important because the two domains are fundamental in high-performance data processing. Deep learning is a current data-intensive workload that has significant industrial and research applications, and molecular modeling is a classical scientific computing workload. Collectively, these examples show that CUDA contributes to both modern AI applications and traditional high-performance simulation by accelerating workloads that depend on large-scale parallel computation.

As demonstrated in Figure 2, CUDA is not just a hardware-based parallel programming model, but a larger software ecosystem. This integration contributes to its popularity in machine learning, scientific computing, and industrial applications because developers can use CUDA together with established frameworks and libraries rather than building GPU programs entirely from scratch.

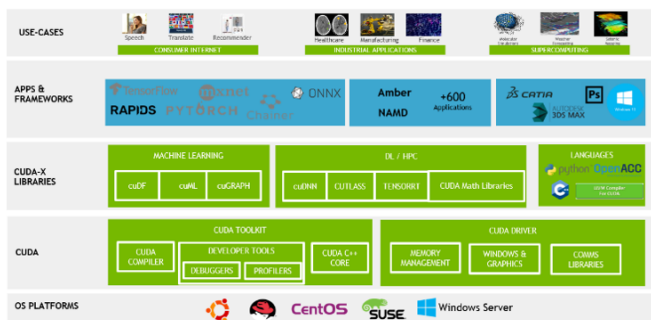


Fig. 2. Overview of the NVIDIA CUDA software ecosystem. Adapted from “What Is CUDA?,” by F. Oh, 2012, NVIDIA Blog.

Nevertheless, the success of these applications do not under cover the limitation of CUDA. Even in scientific computing, Stone et al. (2007) noted hardware restrictions, synchronization challenges, and broader programming complexity. Similarly, in deep learning, the success of GPU training is still constrained by factors such as GPU memory limits and the

TABLE I
COMPARISON OF CPU COMPUTING AND CUDA-BASED GPU COMPUTING

Criteria	CPU Computing	CUDA-Based GPU Computing
Processing orientation	Latency-oriented; optimized to complete individual or sequential tasks quickly	Throughput-oriented; optimized to execute a very large number of parallel operations efficiently
Core design	Fewer but more powerful cores	Many simpler cores designed for parallel execution
Programming model	Conventional sequential programming or limited multi-threading	Explicit parallel decomposition using threads, thread blocks, and grids
Memory behavior	Strong cache hierarchy; often better for reuse-heavy and control-intensive tasks	High memory bandwidth; performs best when memory access is regular and parallelism is abundant
Best suited workloads	Sequential, irregular, branch-heavy, or control-dominant tasks	Highly parallel, arithmetic-intensive, and data-parallel workloads
Performance advantage	Better single-thread and general-purpose flexibility	Higher throughput for deep learning, simulation, and other massively parallel workloads
Ease of development	Easier and more mature programming ecosystem	More complex due to thread coordination, synchronization, and memory management
Major limitations	Lower throughput on large-scale parallel workloads	Sensitive to branch divergence, transfer overhead, and some cache/reuse-heavy workloads

complexity of scaling very large models. Therefore, Subtopic 2 supports the conclusion that CUDA is highly effective in important real-world domains, but its benefits remain dependent on architectural fit and programming design.

V. CONCLUSION

This paper has explored NVIDIA CUDA as one of the significant technologies in high-performance data processing using a systematic comparison with conventional CPU computing. The review of the literature revealed that CUDA was a product of the overall development of GPU computing and provided parallel GPU programming with more convenient abstractions, including threads, blocks, and grids. It also determined that the most successful use of GPU computing is where a workload has a high amount of data parallelism and arithmetic intensity.

The initial subtopic showed that CUDA-based GPU computing is fundamentally different from CPU computing in terms of architectural orientation. CPUs are developed to have low-latency and versatile sequential processing, whereas GPUs are developed to have high-throughput parallel processing. Consequently, CUDA-capable GPUs can frequently provide significantly greater performance on deeply parallel workloads, but this benefit fades away when it comes to irregular or control-intensive or reuse-sensitive computations.

The second subtopic revealed that CUDA has been significantly used in real-life applications in deep learning and scientific computing. The AlexNet breakthrough demonstrated that with the use of a GPU accelerator, it was feasible to train large-scale neural networks, and molecular modeling experiments demonstrated that CUDA can provide significant speedups in scientific work.

On the whole, the key conclusion of the paper is that CUDA is an efficient accelerator of high-performance data workloads, which can be broken down into large numbers of parallel operations. Nonetheless, CUDA cannot be viewed as a universal substitute for CPU computing. Its performance

is determined by the workload organization, memory characteristics, and the complexity of the programming. Further advancements in the architecture of GPUs, memory systems, and programming tools are expected to further enhance the capabilities of CUDA, although CPU computing will still be needed in most areas of general-purpose computation.

REFERENCES

- Fatahalian, K., Sugerma, J., & Hanrahan, P. (2004). Understanding the efficiency of GPU algorithms for matrix-matrix multiplication. In *Graphics Hardware 2004* (pp. 133–137). Eurographics Association.
- Garland, M., & Kirk, D. B. (2010). Understanding throughput-oriented architectures. *Communications of the ACM*, 53(11), 58–66. <https://doi.org/10.1145/1839676.1839694>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, & K. Q. Weinberger (Eds.), *Advances in neural information processing systems* (Vol. 25, pp. 1097–1105). Curran Associates, Inc.
- Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable parallel programming with CUDA. *Queue*, 6(2), 40–53. <https://doi.org/10.1145/1365490.1365500>
- Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., & Phillips, J. C. (2008). GPU computing. *Proceedings of the IEEE*, 96(5), 879–899.
- Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1), 80–113.
- Stone, J. E., Phillips, J. C., Freddolino, P. L., Hardy, D. J., Trabuco, L. G., & Schulten, K. (2007). Accelerating molecular modeling applications with graphics processors. *Journal of Computational Chemistry*, 28(16), 2618–2640. <https://doi.org/10.1002/jcc.20829>